

ME 469: Machine Learning and Artificial Intelligence for Robotics

Assignment 2: Machine Learning

By: Tomasz Krzeminski
Date: 11/17/2025

1 Part A

1.1 Learning Aim (Step 1)

The most common and effective choice of algorithm for path planning is the A* algorithm. To make it more applicable in robotics, variants like Online A* or LRTA* exist to allow the path planning to occur during runtime. The major benefit of A* is that it offers optimal paths as long as the heuristic function is admissible. However, optimal paths are not always beneficial in the scope of a robotics navigation problem. If, for example, the world/space that the robot has to traverse is densely populated with obstacles, and the motion of the robot is very noisy, optimal paths generated by A* can be challenging to traverse. One reason for this is that oftentimes, the shortest paths hug the perimeters of obstacles, as is seen in the left plot of Figure 1. When such a path is then utilized for control of a robot with noisy motion dynamics, the robot can end up colliding with the obstacles or it won't be able to follow that path. As such, these paths require preprocessing and smoothing to make them feasible for robot control, however, even with this optimization, paths that closely follow obstacles will still cause the robot to collide with obstacles as is seen in the right plot of Figure 1. Another downside of the A* algorithm is that it only works in a static map, and has to replan the path if any changes are made. Lastly, the algorithm inherently has no memory of its actions, and has to be called each time that a path needs to be planned.

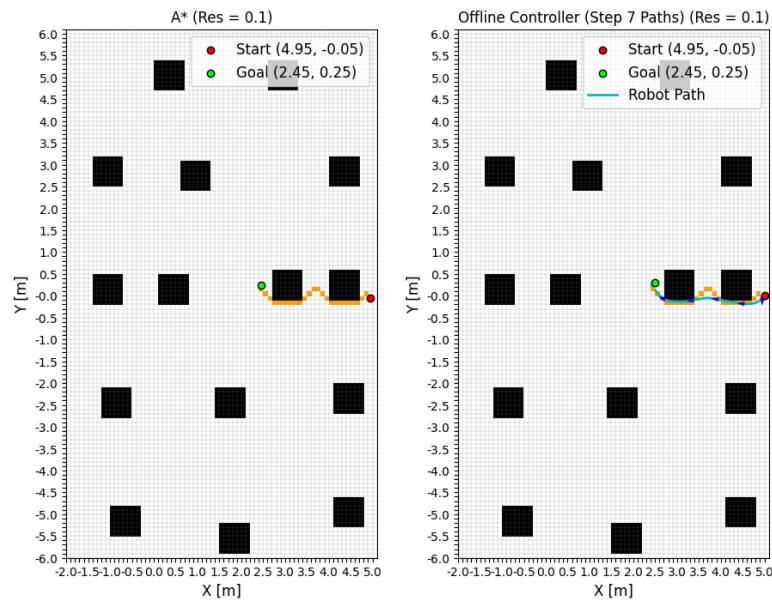


Figure 1: A* Path (Left) and Commanded Robot Trajectory (Right)

In order to improve the capabilities and robustness of path planning for robotics, a machine learning algorithm can be employed. More specifically, the learning algorithm will have to plan optimal paths such as those generated by the A* algorithm. To further expand the capabilities of the algorithm, it will have to be able to learn how to avoid hugging obstacles to make the path more suitable for a robot with noisy motion. It will also need to be capable of “memorizing” the representations of its state space, and correspondingly adjust to any changes made to its environment.

1.2 Learning Algorithm (Step 2)

The best learning algorithm for this learning aim is Q-Learning, which can be justified by analyzing the state space and learning aim requirements. Firstly, Q-Learning stores information about state-action pairs

in a Q-Table, which is a tabular representation of a state space, in which each cell of the table is a state-action pair value represented by a corresponding Q-Value (Thakur, 2023). As such, a prerequisite for this algorithm is that the state space and action set can be discretized. However, it should also be noted that the discretization is not too fine in order to reduce computational and memory complexity.

In the case of this problem, the state space, represented by the grid world, can be discretized as is evident by the fact that each state is represented by a cell in the grid world. Furthermore, the action set for this world is simply the set of 8-Neighbors around the robot. In addition to this, the discretization of the world can be analyzed for both the 1.0m and 0.1m resolutions of the grid world. From this analysis, it can be found that the 1.0m resolution grid world has a height of 12 and a width of 7, resulting in 84 states that need to be represented in the Q-Table. The 0.1m resolution is considerably larger with a height of 121 and a width of 71, resulting in a total of 8591 states that need to be represented. Despite this, this number of states is still computationally tractable making this algorithm viable for the learning objective.

Another requirement of the Q-Learning algorithm is that the environment that it operates around follows the Markov Property (Thakur, 2023). This means that the future state solely depends on the current state and action (i.e. it is independent of all other state-action pairs required to get to the current state). Considering that the grid world is static, the current state of the robot fully defines the state representation, requiring no prior knowledge to have an understanding of that state. Furthermore, the actions in the action set are all deterministic, meaning that even after state transition, the state is fully known. For these reasons, our data follows the Markov property, making it suitable for Q-Learning.

Lastly, because of the fact that each action is deterministic (i.e. taking the same action from the same state will always result in the same next state) and because there is a finite number of states, it is very likely that Q-Learning converges. Kaelbling (1996) states that if each action is performed an infinite number of times, and the learning rate is decayed correspondingly, the Q-values will eventually converge to the optimal policy $\pi^*(s)$. Considering that there are a finite number of states, the algorithm should be able to converge to an optimal policy as long as it is properly designed.

1.3 Problem Formulation (Step 3)

The learning aim of this assignment is to learn a path for robot navigation in a 2D grid world populated with obstacles. As such, the learning agent aims to learn a policy that reaches a goal position, avoids collisions and maintains a safe distance from obstacles, as described in section 1.1. The number of states for each resolution is given by Table 1.

Resolution	Grid Dimensions	Number of States
1.0 m	7 x 12	84
0.1 m	121 x 71	8591

Table 1: State-Space Size for Each Grid Resolution

Problem Space:

- State (s_t): (x, y) – Position in Grid World
- Action Set (A_t): $\{(-R, 0), (R, 0), (0, R), (0, -R), (-R, R), (R, R), (R, -R), (-R, -R)\}$, $R = \text{grid resolution}$
- Transition (s_{t+1}): $s_t + a_t$

The problem space is defined by the state at which the robot is, the actions it can take as defined by the action set, and the transition from one state to another. The state in this scenario is simply the robot

position in the grid world. The action set is defined by the 8-Neighbors actions for a grid world, and the distance that the agent moves is equal to the resolution of the grid world. The transition is simply defined to be the current position of the robot plus whichever action is chosen from the action set.

Reward Function:

$$x_t = \text{Current State} \quad (1)$$

$$d_{goal} = \sqrt{(x_g - x_t)^2 + (y_g - y_t)^2} \quad (2)$$

$$d_{obs} = \sqrt{(x_{obs} - x_t)^2 + (y_{obs} - y_t)^2} \quad (3)$$

$$\text{Reward}(r_t) = \begin{cases} 10, & d_{goal} = 0 \\ -10, & d_{obs} = 0 \\ -\beta d_{goal}, & \forall x_t \\ -\delta d_{obs}, & d_{obs} < 1 \end{cases} \quad (4)$$

The reward function, as seen in equation (4), is designed to generate a large positive reward whenever the robot reaches the goal position, and a large negative reward whenever the robot collides, or is inside an obstacle. The first reward incentivizes actions around the goal, by propagating the positive reward for each action that moves towards the goal. The second reward disincentivizes actions around any obstacle, by propagating a negative reward for each action that can result in the robot moving into an obstacle. The third and fourth rewards in the reward function serve as attraction and repulsion gradients. The third reward assigns a negative reward to the current state depending on how far it is from the goal position, which is scaled by a weight. Overtime, this creates a Q-Value gradient that attracts the robot to the goal position. The fourth reward assigns a negative reward to the robot if the robot is within 1 unit to any obstacle. This results in the robot learning to stay at least one tile away from an obstacle instead of hugging it.

Policy Definition:

Training Policy (ϵ -Greedy):

$$a_t = \pi^*(s) = \begin{cases} \text{random}(a_t), & P(\epsilon) \\ \text{argmax}_{a_t} Q^\pi(s, a), & P(1 - \epsilon) \end{cases} \quad (5)$$

Testing Policy (Greedy):

$$a_t = \pi^*(s) = \text{argmax}_{a_t} Q^\pi(s, a) \quad (6)$$

The training policy is dictated by the ϵ -Greedy policy, as seen in equation (5), which selects the next action based on the selection probability, which is defined by ϵ . If $P(\epsilon)$ is greater than $P(1 - \epsilon)$, the policy is updated by selecting a random action from the action set. If the opposite is true, the best action is chosen by selecting the action from the state-action pair with the largest Q-Value. For testing, the update is dictated by a Greedy policy, as seen in equation (6), which simply selects the best action from the state-action pair with the largest Q-Value.

Q-Value Update Function:

$$Q(s, a) = Q(s, a) + \alpha[r_t + \gamma Q(s', a') - Q(s, a)] \quad (7)$$

The Q-Value update function, as seen in equation (7), updates the Q-Value by adding on a temporal difference (TD) error, which is scaled by a learning rate. The temporal difference error is computed by summing the reward, which was sampled at the current state, the discounted estimate of the optimal Q-Value at the next state, and the Q-Value at the current state (Thakur, 2023).

Tuneable Parameters:

Parameter	Description	Standard Value
ϵ	Exploration Parameter	0.3
α	Learning Rate	0.8
γ	Discount Factor	0.95
β	Distance to Goal Reward Weight	0.1
δ	Distance to Obstacle Reward Weight	0.5
Episodes	Training Episodes	3000 - 10000

Table 2: Tuneable Parameters for the Q-Learning Problem

The tuneable parameters for Q-Learning are listed in Table 2. The exploration parameter, denoted by ϵ , dictates the ϵ -Greedy policy overviewed above. This parameter will be tuned by observing how dispersed the Q-Table values are at the end of the training, and by how quickly the training converges to an optimal path. The learning rate, denoted by α , dictates how much the Q-Value is changed each episode. Larger values will result in more drastic value changes, which speeds up how fast the Q-Table changes. This can hurt convergence if the value is too large. This parameter will be tuned by observing how fast the agent is able to converge to the ideal path, and how stable the convergence is. The discount factor, denoted by γ , dictates how much impact the estimated future Q-Value will have on the new Q-Value. This will be tuned by observing the convergence of the Q-Values. The distance to goal reward weight, β , dictates the attractive pull of the goal position on the reward value. This will be tuned similarly to the exploration parameter. Lastly, the distance to obstacle reward weight, δ , dictates whether the robot learns to not hug the obstacles. This parameter will be tuned by increasing the value until the robot no longer hugs obstacles.

1.4 Q-Learning Overview (Step 4)

The Q-Learning algorithm is a model-free algorithm, meaning that it executes actions multiple times, modifying the policy each time in order to maximize the reward at each state (Thakur, 2023). It is also an

off-policy algorithm, which means that it uses separate policies for control and evaluation. This algorithm learns a state-action value function ($Q(s,a)$), which estimates the temporal difference of taking an action in a current state and then following that action.

During training, the actions are selected by means of the ϵ -Greedy policy (5), which allows the agent to explore or exploit its actions depending on the value of ϵ . The agent then performs a state transition by moving with the selected action, and samples the reward at the new state through the reward function (4). The Q-Value is then updated through the Q-Value update function (7). This process is repeated for the specified number of episodes. Over the course of all the episodes, the Q-Table will begin to converge to a stable estimate of the state-action representation. With the optimal values, the Greedy policy (6) can be used to extract the optimal path. The training loop for this process can be seen in Figure 2.

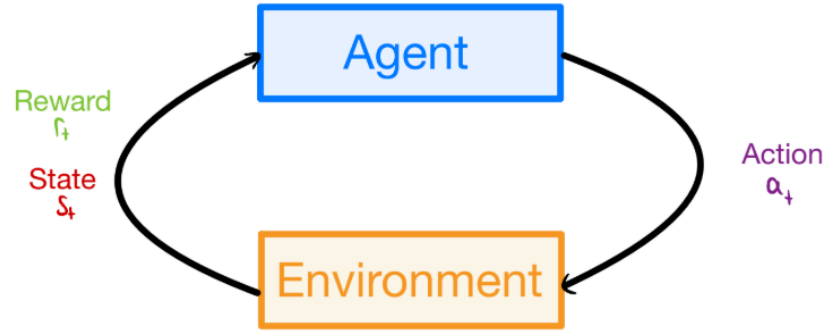


Figure 2: Training Loop For Q-Learning Agent

1.4.1 Simple Test (Step 5)

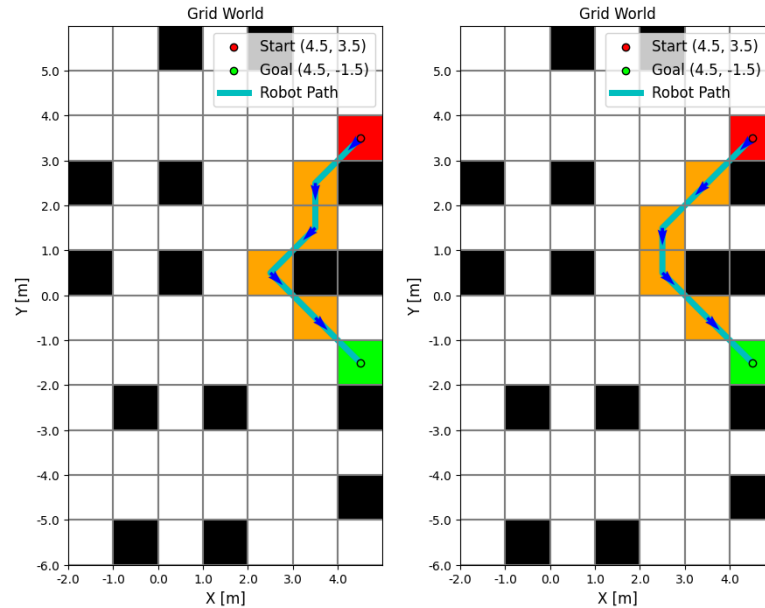


Figure 3: Optimal Path (Left) and Obstacle Avoidance Path (Right) Generated by Q-Learning in Coarse Grid World

From Figure 3 it can be seen that the Q-Learning algorithm was able to generate an optimal path as is evident from the left plot, and an optimal path which avoids hugging the obstacle in the right plot. The

obstacle clearance in this example is less evident due to low resolution of the grid, but it is clear that the agent was able to learn a behavior which avoided following along the perimeter of an obstacle, while still being able to follow an optimal path. Nonetheless, the agent was able to find an optimal path from a start to goal position.

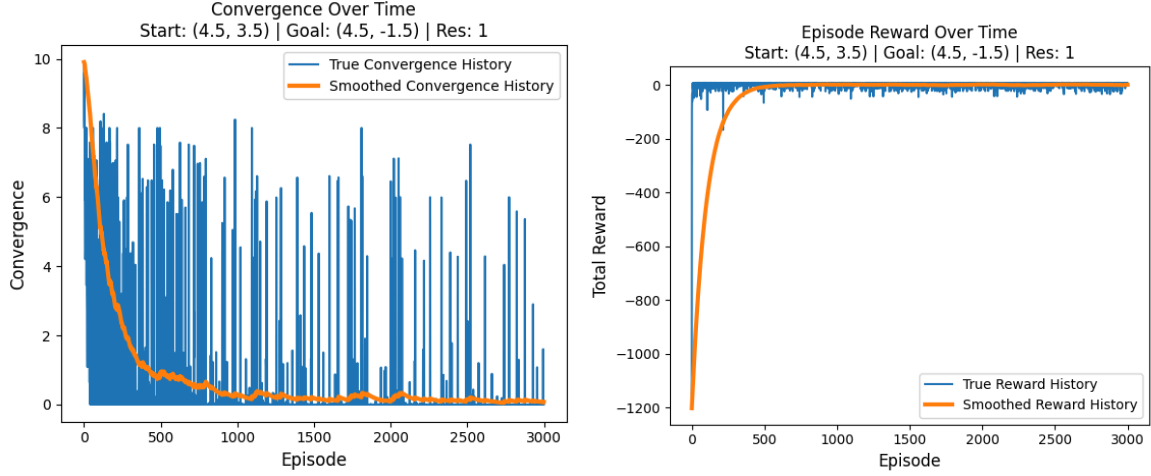


Figure 4: Convergence History (Left) and Reward History (Right) for Optimal Path (Left Plot in Figure 3)

From the left plot in Figure 4, it can be seen that the agents converged to an optimal path as the orange line, which represents a filtered convergence history, converges to a steady state position near zero. The noise in the unfiltered convergence history occurs because the agent continues to explore the environment, leading to large changes in the Q-Table. This convergence history would likely stabilize only if we decay the exploration parameter over time so that the agent purely exploits as the episode count grows. If we were to increase the number of episodes, more states would be explored, but the noise in the convergence history would remain, due to the fact that the robot can still explore. This would lead to faster convergence, but would result in a less explored state space. The reward history shows that the agent was able to find an optimal path that maximizes the cumulative reward of each episode. This is evident due to the fact that the cumulative reward remains the same across the episodes.

2 Part B

2.1 Learning Paths (Step 6)

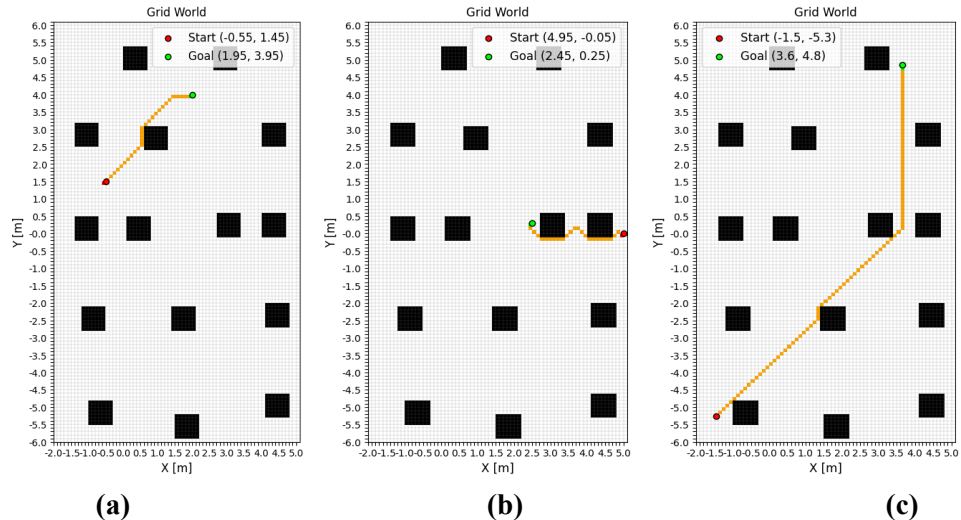


Figure 5: Optimal Paths Obtained from Greedy Policy Q-Learning for High Res. Grid

The first performance metric that needs to be evaluated is the optimality of the paths generated by the algorithm. From Figure 5, it can be seen that the paths without collision avoidance are optimal, as the path length is the shortest possible path. The robot trajectory is omitted from this example, and only the path is plotted for ease of viewing. Nonetheless, this result shows that the algorithm is capable of learning optimal paths. Next, we will see if the algorithm is capable of finding optimal paths that avoid obstacle perimeters.

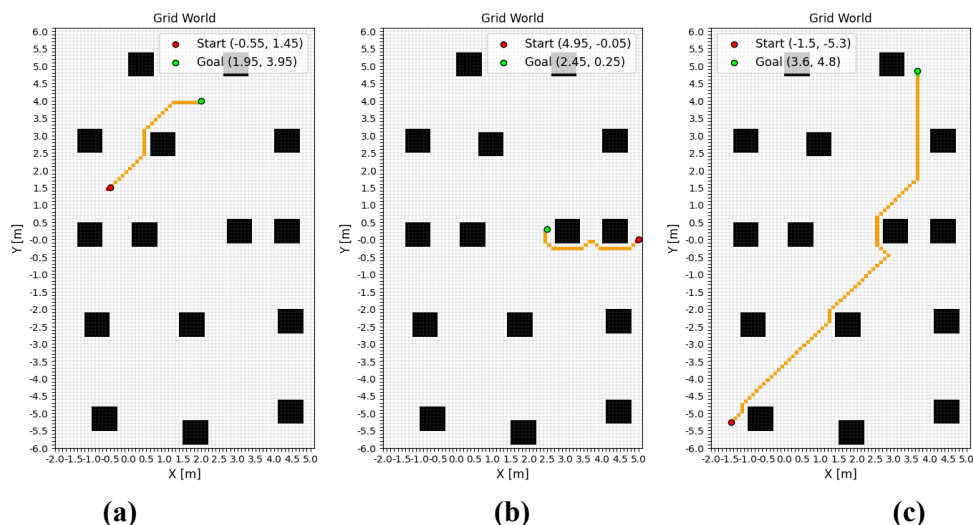


Figure 6: Obstacle Avoidance (OA) Paths Obtained from Greedy Policy Q-Learning for High Res. Grid

From Figure 6 it can be seen that the algorithm was able to find optimal paths and explicitly avoid following the edges of the obstacles. This is most evident in plot (c) which resulted in the algorithm finding a completely new optimal path. It was able to learn this behavior thanks to the specified reward function, which punished the agent whenever it was close to the obstacles. Given these results, it can be concluded that the Q-Learning algorithm is capable of finding optimal paths with and without obstacle avoidance.

2.1.1 A* Paths v.s Q-Learning Paths

To compare the performance of the Q-Learning algorithm with the performance of the A* algorithm, the problem definition for plot (b) from Figure 6 will be assumed.

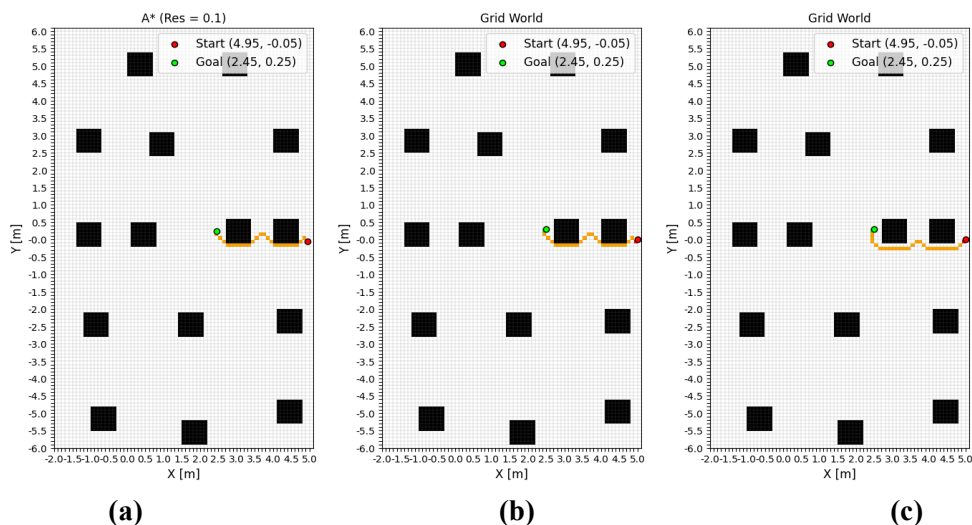


Figure 7: A* Path (a), Optimal Path (b) and OA Path (c) Assuming Same Start, Goal and Resolution

From Figure 7, it can be seen that plots (a) and (b) are exactly the same. This further supports the fact that the Q-Learning algorithm was able to find an optimal path as the A* formulation resulted in an optimal path. Furthermore, it can be seen that the Q-Learning algorithm was able to find a path that focused on obstacle avoidance. Now we will compare QL + PID to A* + PID for the obstacle avoidance path and the A* path to observe the impact of obstacle avoidance. The robot dynamics will assume the noise parameter, α , to be 0.5.

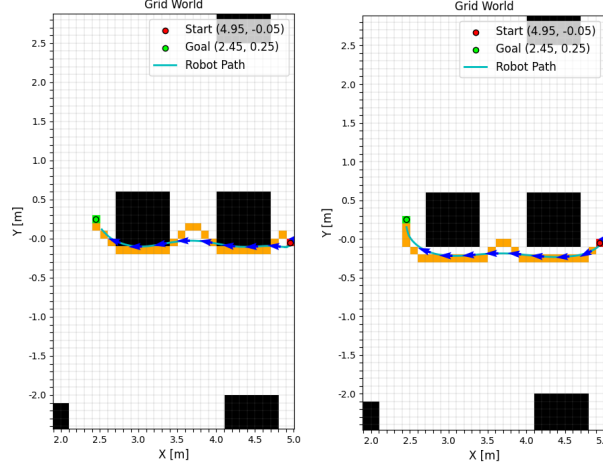


Figure 8: A* + PID (Left) and OA + PID (Right)

The results seen in Figure 8 show that the A* + PID robot path had a smooth control path, however, it collided with both obstacles due to the fact that the A* path followed the edges of the obstacles. This issue is mitigated in the OA + PID control, as the robot was able to follow the control path, and completely avoid collisions with the obstacles. This result further supports that Q-Learning is able to find optimal paths like A*, however, it is also able to learn paths that avoid following obstacles.

2.1.2 Q-Table Visualization

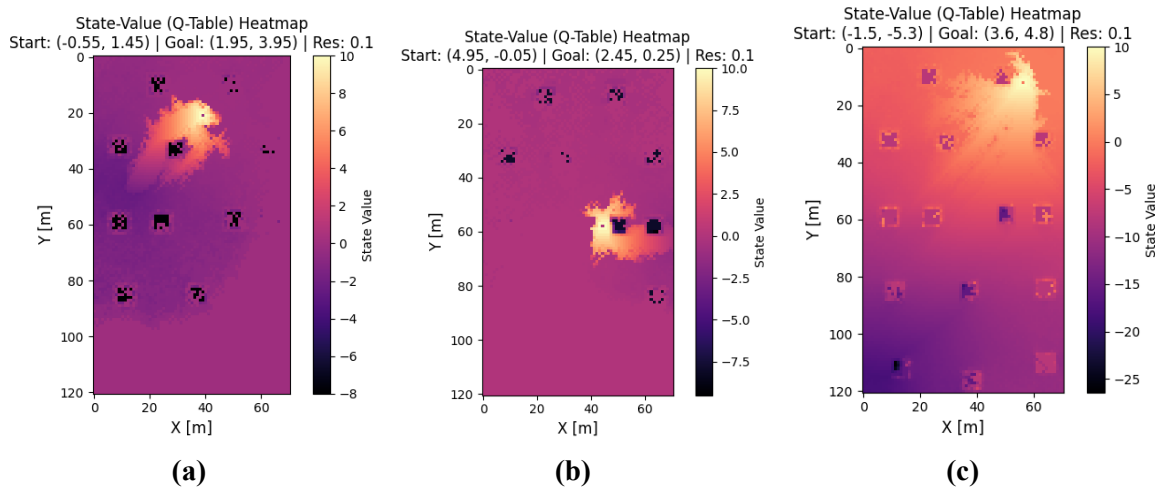


Figure 9: Heatmap Visualization of Q-Tables for Plots from Figure 6

The following visualization of the Q-Tables for each of the OA paths from section 2.1 shows that the Q-Tables for (a) and (b) are less filled, and focused more on exploitation. The most likely reasons for this result are the choice of ϵ and the number of episodes, as these two plots both have $\epsilon = 0.3$, and an episode count of 3,000. Plot (c) has a more defined Q-Table, however, the training ran for 10,000 episodes. This result suggests that performing training for a larger number of episodes will better populate the Q-Table.

Furthermore, these plots show that the regions around the goal position propagate the reward outward, resulting in an attractive gradient of Q-Values, whereas the obstacles are repulsive, resulting in the negative Q-Values that disincentivize actions to those states.

2.1.3 Path Performance Metrics

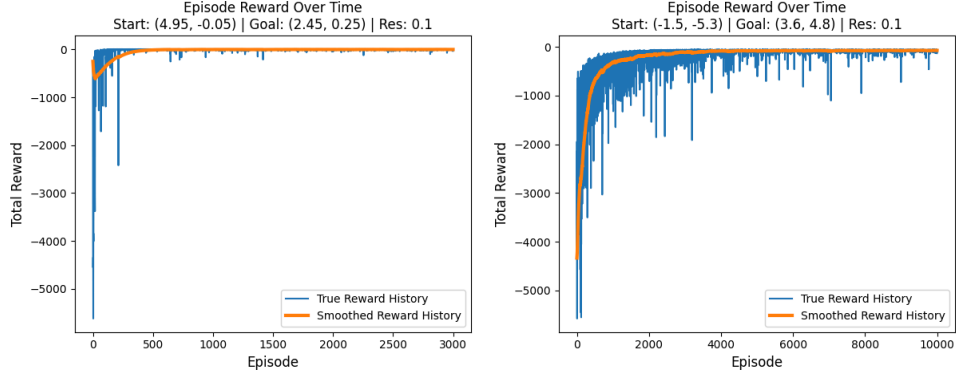


Figure 10: History of Cumulative Rewards per Episode of Plots (a) and (c) from Figure 6

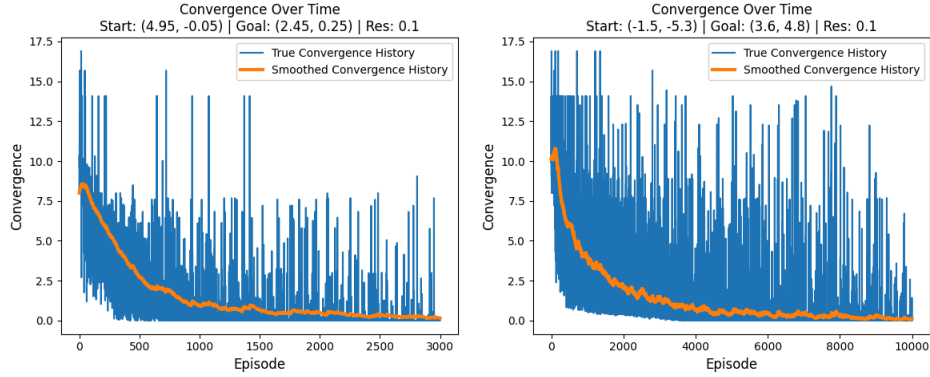


Figure 11: History of Convergence for Plots (a) and (c) from Figure 6

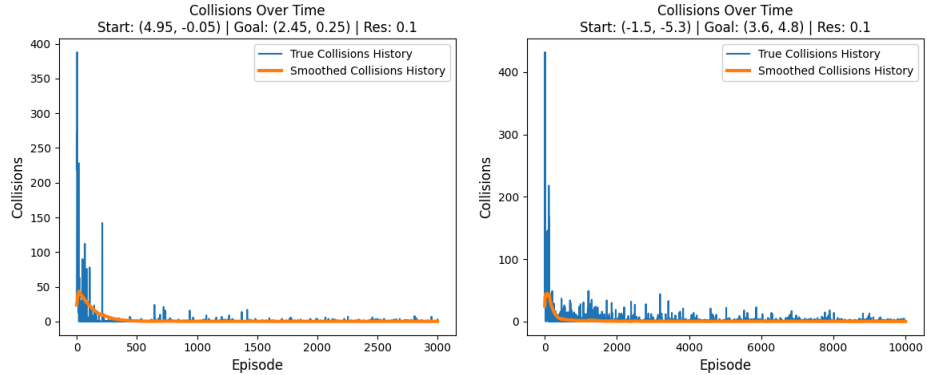


Figure 12: History of Collisions per Episode of Plots (a) and (c) from Figure 6

The following figures show the cumulative reward histories, convergence histories, and collision histories of the corresponding plots of the OA paths in Figure 6. From these plots it can be seen that for short paths, the cumulative reward and convergence settle fast, as the training only lasts for 3,000 episodes. Furthermore, the amount of noise in the plots for the shorter path is a lot less than the amount of noise in the longer path. The reason for this difference is that there are a lot more states for the longer path to explore before it finds a path from the start to the goal position. Because of this, it takes a lot longer for the algorithm to converge to an optimal path, as is evident by the plot in Figure 11. Figure 12 showcases the collision

history of both paths, and it can be seen that the agent is able to learn to avoid collisions very fast, taking roughly 300 episodes for the shorter path, and 1000 episodes for the longer path. The longer path experiences more collisions for a longer period after this, however, this is expected as there are a lot more states for the agent to get to the goal position.

2.2 Discussion

Overall, the learning algorithm performed very well. Firstly, it was able to recover optimal paths when obstacle avoidance was omitted. This means that the reward function simply reduced to $r_t = -\beta d_{goal}$, which is simply the scaled heuristic distance seen in the A* algorithm. This would then result in a smooth distance-to-state gradient propagation defined by the Bellman optimality equation (8).

$$Q^*(s, a) = r_t + \gamma \max_a Q^*(s', a) \quad (8)$$

This is exactly the result we see in the plots of Figure 9, and the optimal paths of Figure 5. This behavior is further expanded upon when obstacle avoidance is introduced in the reward function through $r_t = -\beta d_{goal} - \delta d_{obs}$, which adds a repulsive potential around the obstacle. The Bellman optimality equation then propagates these negative rewards through the Q-Values, resulting in regions of negative Q-Values that disincentivizes the robot from those states. Lastly, the quick falloff in the number of collisions is also a good behavior, as the agent quickly learns that obstacles collisions are bad. This occurs because the temporal difference update in equation (7) propagates the collision penalty $r_t = -10$, into the states with obstacles. States with actions leading to the obstacles then quickly acquire low Q-Values, which makes the *Greedy* policy less likely to choose them.

Despite the good results, however, there exist some poor behaviors. Firstly, the algorithm converges a lot slower for larger state spaces as is seen in Figure 11. This occurs because the number of state-action pairs that need to be visited and updated through equation (7) grows larger. Another negative behavior seen in the results is the sparse Q-Tables seen in Figure 9. This likely occurs for a number of reasons. One reason that this could occur is that the choice of $\epsilon = 0.3$ is simply too low, resulting in more exploitation than exploration. Another likely reason for this result is that the greedy policy selection in the ϵ -*Greedy* policy selects states that have been visited. As such, with small amounts of exploration, the policy will repeatedly choose the same actions. To address this issue, a larger ϵ can be chosen, and the training can be run for more episodes. Nonetheless, the results seen for this algorithm show that it is a viable option when planning paths with obstacle avoidance.

References Cited

- Thakur, P. (2023, September 27). *What is Q-learning?*. W&B.
<https://wandb.ai/cosmo3769/Q-Learning/reports/What-is-Q-Learning---Vmldzo1NTI1NzE0>
- Kaelbling, L. P. (1996). Q-learning.
<https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume4/kaelbling96a-html/node25.html>